

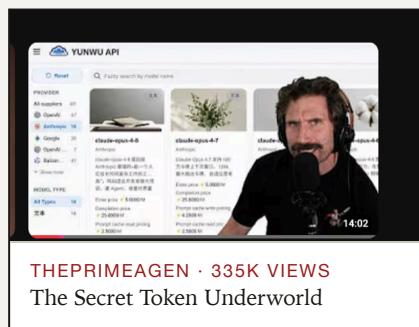
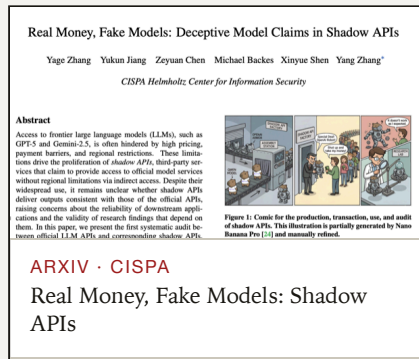
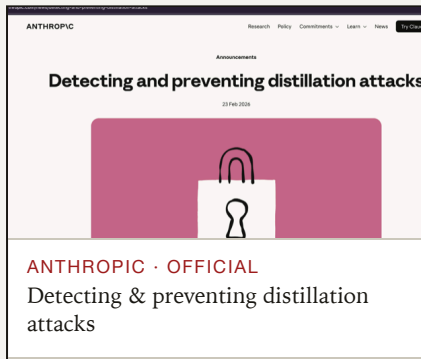
The ProofOfModel

MONAD BLITZ PUNE · VOL. II BACKGROUND → PROBLEM → SOLUTION TESTNET · CHAIN 10143

RECENTLY SURFACED · FEB-JUL 2026

Small today. A big problem tomorrow.

Not mainstream yet — but in just a few months a model lab, security researchers, and the developer community have each independently flagged it. As the Agent Economy scales, this becomes a major issue. We got to it early.



A market for AI tokens you weren't meant to see

Frontier models are metered by the token. Where they're blocked or unaffordable, a grey economy resells that metering — and it has grown into an industrial supply chain.

\$15

ANTHROPIC · 1M OPUS TOKENS



\$1

BLACK-MARKET RESELLER

The math is impossible — unless the model is swapped, the credential is stolen, or your prompts are resold. **Usually all three.**

70–95%

below official price

45.8%

of shadow APIs lie about
the model

83.8→37

Med-QA accuracy when
swapped

17 / 187

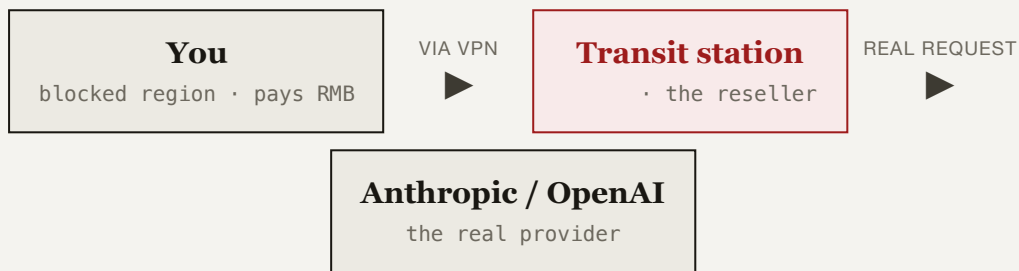
fake APIs found in
published papers

Two harms hide behind one cheap proxy

Resellers run "transit stations" (中转站). The first harm is what they sell you; the second is what they quietly take. Here is how each works.

FIGURE 1 · THE HARM YOU CAN'T SEE

Anatomy of a transit station



↪ returns a **cheaper model** than you paid for — you ask Opus, it serves Haiku / Kimi **MODEL SWAP**

↑ FED BY A FRAUD SUPPLY CHAIN ↑

Stolen cards

free Max plans

SMS / SIM farms

~\$0.008 / number

AI fake IDs

\$15 · passes KYC

Pooled accounts

shared Max limits

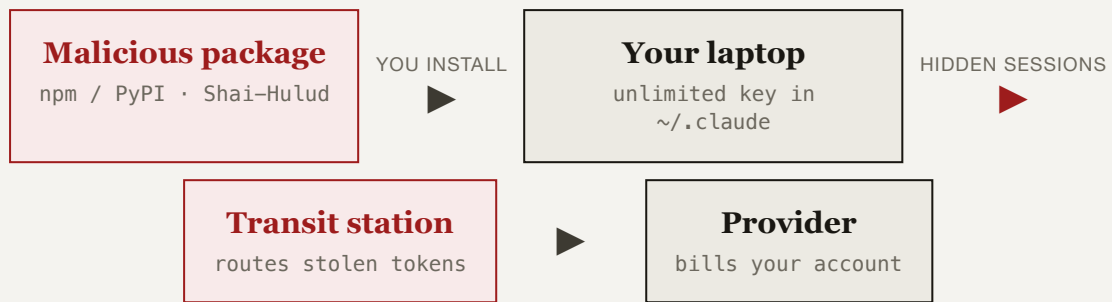
↓ AND IT SKIMS THE REAL GOLD ↓

Your prompts + the model's reasoning are logged and **resold to Chinese labs as training data**. The token discount is the loss-leader; your data is the product.

Modular by design: kill one SMS or ID supplier and the station keeps running — the supply side is practically un-killable.

FIGURE 2 · THE HARM THAT EMPTIES YOUR ACCOUNT

How a worm steals your tokens



The worm sips your quota to stay invisible. All **you** see is
WEEKLY LIMIT HIT — millions of hidden requests a day, on machines
whose owners have no idea.

Root cause: an unlimited, long-lived bearer credential sitting on a machine the malware now controls.

Two modules, one trust layer

MODULE I · AUTHENTICITY

Proof of Model

Fingerprints which model an endpoint really serves, then writes the verdict + a reputation score to Monad.

- Self-ID, refusal style, reasoning quirks, tokenizer tells
- Cheating endpoints earn a permanent on-chain red flag
- Any agent checks reputation before it pays

MODULE II · ANTI-THEFT

Prepaid Gateway

Replaces the stealable key with scoped, prepaid, per-call access — settled on Monad.

- Real provider key held server-side; agents never touch it
- Session = spend cap + expiry + instant revoke()
- Every inference debited on-chain — auditable, bounded

✗ Today: an unlimited bearer key

A long-lived key sits in ~/.claude. A worm reads it and drains your whole quota — silently, unbounded.

Nothing caps it. Nothing shows it. Nothing revokes it.

✓ Ours: a scoped session

The real key lives server-side. All a worm finds is a capped, expiring, revocable session — every use an on-chain debit.

Theft becomes bounded and visible, not silent and unlimited.

FIGURE 3 · THE FIX

Where ProofOfModel cuts each loop

CUTS FIGURE 1

Local trust proxy

Sits between you and the station; verifies the endpoint + its on-chain reputation and **blocks the model swap** before your request leaves the laptop. Flags "this is a transit station, not the provider."

CUTS FIGURE 2

Prepaid Gateway (on Monad)

The real key lives off your machine, so the worm can't steal it. It finds only a **capped, expiring, revocable** session — every use an on-chain debit you can see.

The trust & settlement layer

Everything requiring trust between parties who don't trust each other lives on-chain. Fingerprinting stays off-chain — it must call the models.

ON-CHAIN	CONTRACT CALL	WHY A CHAIN, NOT A DATABASE
Reputation ledger	<code>attest · reputation</code>	Public & tamper-proof — a cheating proxy can't erase its history
Prepaid balance	<code>deposit · withdraw</code>	Self-custody — you control funds and withdraw anytime
Scoped session	<code>openSession · revoke</code>	Cap / expiry / revoke enforced by code — the operator can't overcharge
Per-call settlement	<code>debit</code>	Public metering — every charge is an auditable event

ETHEREUM L1		MONAD	
Block time	~12 s	Block time	~1 s
Per-call gas	> token cost	Throughput	~10,000 TPS
Micropayments	impossible	Micropayments	viable

Monad's throughput is the enabling condition: per-call on-chain LLM billing is only possible on a fast, near-free chain.

What we neutralize

ATTACK	NEUTRALIZED BY	STATUS
Shadow-API model swap	Authenticity oracle + on-chain reputation	SOLVED
Worm hijacks subscription	Scoped session: cap + expiry + revoke	SOLVED
Stolen or leaked API key	Provider key held server-side	SOLVED
Account or Max pooling	Per-call on-chain metering + attribution	SOLVED
Stolen credit cards	Prepaid balance replaces card billing	SOLVED
Prompt data harvesting	First-party gateway; custody proof next	PARTIAL

Three moves

I Catch the swap

Verify an endpoint claiming `claude-opus-4.8` → authentic, recorded on Monad. Point it at a proxy secretly serving GPT or Kimi → substitution detected, reputation drops.

II Meter a real call

Deposit MON, open a small-cap session, call real Claude through the gateway — debited live on Monad.

III Pull the kill switch

Revoke the session; the next call is blocked on-chain. That is the worm defense — bounded and revocable.

Live on Monad Testnet

AttestationRegistry · <i>authenticity oracle</i>	0x21f082...a0a1d92 ↗
PrepaidGateway · <i>secure access</i>	0x7ed90e...73a6ec ↗
Sample attestation transaction	0x397631...b8de851 ↗
Sample per-call debit transaction	0x079c35...010a3e9d ↗

Validated live against a real LiteLLM proxy fronting Anthropic, Azure-OpenAI and Moonshot. Built with Next.js, viem and solc.